1. Generative AI: Prompt Engineering Basics

- Text prompts can improve the reliability and quality of the output generated from LLMs with techniques such as:
    - Task specification
    - Contextual guidance
    - Domain expertise
    - Bias mitigation
    - Framing
    - User feedback loop
- Zero-shot prompting refers to the capability of LLMs to generate meaningful responses to prompts without needing prior training.
- Few-shot prompting used with LLMs relies on in-context learning. In this technique, demonstrations are provided in the prompt to steer the model toward better performance.
- Using text prompts with LLMs effectively has several benefits, including increasing their explainability, addressing ethical considerations, and building user trust.
- The interview pattern is superior to the conventional prompting approach as it allows for a more dynamic and iterative conversation when interacting with generative AI models.
- Chain-of-Thought strengthens the cognitive abilities of generative AI models and solicits a step-by-step thinking process.
- Tree-of-Thought is an innovative technique that builds upon the Chain-of-Thought approach and involves structuring prompts hierarchically, akin to a tree, to guide the model's reasoning and output generation.

Term	Definition
Application programming interface (API) integration	The process of connecting different software systems or applications through their APIs to enable them to work together and share data or functionality.
Bias mitigation	A technique in which text prompts provide explicit instructions to generate neutral responses.

Chain-of-Thought	An approach to prompt engineering that involves breaking down a complex task into smaller and easier ones through a sequence of more straightforward prompts.
ChatGPT	A language model designed to provide detailed responses to natural language input.
Claude	A powerful and flexible AI chatbot to help you with your tasks.
Comparison prompting	A technique where the model is asked to evaluate multiple outputs side-by-side to determine which one is better or more accurate.
Contextual guidance	A technique using which text prompts provide specific instructions to the large language models (LLMs) to generate relevant output.
Cross-modal understanding	The ability of AI models to connect and reason across different types of input, like using information from an image to answer a text-based question.
DALL-E	Text-to-image model that generates digital images from natural language descriptions.
Domain expertise	A technique wherein text prompts can use domain-specific terminology to generate content in specialized fields like medicine, law, or engineering.
Dust	A prompt engineering tool that provides a web user interface for writing prompts and chaining them together.
Explainability	Refers to the degree to which a user can understand and interpret the model's decision-making process and the reasons behind its generated outputs.

Few-shot prompting	A method that enables context learning, wherein demonstrations are provided in the prompt to steer the model to better performance.
Framing	A technique by which text prompts guide LLMs to generate responses within the required boundaries.
Generative AI	A type of artificial intelligence that can create new content, such as text, images, audio, and video.
Generative AI models	Models that can understand the context of input content to generate new content. Used for automated content creation and interactive communication.
Generative pre-trained transformers (GPT)	A family of neural networks that use transformer architecture to create human-like text or content as output.
IBM watsonx.ai	A platform of integrated tools to train, tune, deploy, and manage foundation models easily.
Input data	Any piece of information provided as part of the prompt.
Integrated Development Environment (IDE)	A software tool for crafting and executing prompts that engage with language models.
Interview pattern approach	A prompt engineering strategy that involves designing prompts by simulating a conversation or interacting in the style of an interview.
LangChain	A Python library that provides functionalities for building and chaining prompts.

Large language models (LLMs)	Deep learning models trained on massive amounts of text data to perform tasks such as text generation, summarization, translation, and sentiment analysis.
Midjourney	A text-to-image model that generates images from natural language requests.
Multi-modal models	AI models capable of processing and generating multiple types of data such as text, images, audio, and video.
Multi-modal prompts	Prompts that include multiple forms of input, such as text, images, or audio, to guide AI models.
Naive prompting	Asking queries from the model in the simplest possible manner.
Natural language processing (NLP)	A branch of artificial intelligence that enables computers to understand, manipulate, and generate human language.
OpenAI Playground	A web-based tool that helps to experiment and test prompts with various models of OpenAI, such as GPT.
Output indicator	Benchmarks for assessing the attributes of the output generated by the model.
Play-off method	A prompting technique where multiple outputs are generated for the same input, then compared or evaluated to select or refine the best response.
Prompt	Instructions or questions given to a generative AI model to generate new content.
Prompt engineering	The process of designing effective prompts to generate better and desired responses.

Prompt lab	A tool that enables users to experiment with prompts based on different foundation models and build prompts based on their needs.
PromptBase	A marketplace for selling and buying prompts.
PromptPerfect	A tool used to optimize prompts for different LLMs or text-to-image models.
Role-play/Persona pattern	A format or structure for constructing prompts that involve the perspective of a character or persona.
Scale AI	A technology company that specializes in data labeling and data annotation services.
Self-reflection prompting	A technique where the model is instructed to review or critique its own outputs to improve accuracy or creativity. Often paired with the play-off method.
Stable Diffusion	A text-to-image model that generates detailed images based on text descriptions.
StableLM	An open-source language model based on a dataset that contains trillions of tokens of content.
Tree-of-Thought	An approach to prompt engineering that involves hierarchically structuring a prompt or query to specify the desired line of reasoning.
User feedback loop	A technique wherein users provide feedback to text prompts and iteratively refine them based on the response generated by the LLM.

Zero-shot prompting	A method where generative AI models generate meaningful responses to prompts without needing prior training on those specific prompts.

Cheat Sheet: Fundamentals of Building AI Agents using RAG and LangChain

Package/Method	Description	Code example
Generate text	This code snippet generates text sequence	1 2 3 4 5 6 7 8 9 10 11 12 13 • # Generate text • output_ids = model.generate(• inputs.input_ids,

s
b
a
s
e
d
o
n
t
h
e
i
n
p
u
t
a
n
d
d
o
e
s
n
'
t
c
o
m
p
u
t
e
t
h
e
g
r
a
d
i
e
n
t
t
o
g
e
n

```
    attention_mask=inputs.attention_mask,
    pad_token_id=tokenizer.eos_token_id,
    max_length=50,
    num_return_sequences=1
)
output_ids
or
with torch.no_grad():
    outputs = model(**inputs)
outputs
```

Copied!

Wrap Toggled!

	erateoutput.	
formatting_prompts_func_no_response function	The prompt function generates formatted	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15

text prompts from mydataset by using the instructions from

```
def formatting_prompts_func(mydataset):
    output_texts = []
    for i in range(len(mydataset['instruction'])):
        text = (
            f"### Instruction:\n{mydataset['instruction'][i]}"
            f"\n\n### Response:\n{mydataset['output'][i]}"
        )
        output_texts.append(text)
    return output_texts
def formatting_prompts_func_no_response(mydataset):
    output_texts = []
    for i in range(len(mydataset['instruction'])):
        text = (
            f"### Instruction:\n{mydataset['instruction'][i]}"
            f"\n\n### Response:\n"
        )
        output_texts.append(text)
```

	the dataset. It creates strings that include only the inst	• return output_texts Copied! Wrap Toggled!

	ructionandapplaceholderforthereseponse.	

torch.no_grad()	This code snippet helps generate text sequences from the p	```
		with torch.no_grad():
		  # Due to resource limitation, only apply the function on 3 records using "instructions_torch[:10]"
		  pipeline_iterator= gen_pipeline(instructions_torch[:3],
		  max_length=50, # this is set to 50 due to resource constraint, using a GPU, you can increase it to the length of your choice
		  num_beams=5,
		  early_stopping=True,)
		generated_outputs_lora = []
		for text in pipeline_iterator:
		  generated_outputs_lora.append(text[0]["generated_text"])
		``
		1 2 3 4 5 6 7 8 9

	i p e l i n e f u n c t i o n . I t e n s u r e s t h a t t h e g r a d i e n t c o m p u t a t i	Copied! Wrap Toggled!

	onsaredisabledandoptimizestheperformanceandmemory	

	usage.	
mixtral-8x7b-instruct-v01 watsonx.ai inference model object	Adjusts the parameters to push the limits of crea	model_id = 'mistralai/mixtral-8x7b-instruct-v01' parameters = {

tivityandresponselength.

```
    GenParams.MAX_NEW_TOKENS: 256,  # this controls the maximum number of tokens in the generated output
    GenParams.TEMPERATURE: 0.5, # this randomness or creativity of the model's responses
}
credentials = {
    "url": "https://us-south.ml.cloud.ibm.com"
}
project_id = "skills-network"
model = ModelInference(
    model_id=model_id,
    params=parameters,
    credentials=credentials,
    project_id=project_id
)
```

Copied!

Wrap Toggled!

String prompt templates	Used to formatasinglestringandaregenerallyusedforsi	1
		2
		3
		4
		from langchain_core.prompts import PromptTemplate
		prompt = PromptTemplate.from_template("Tell me one {adjective} joke about {topic}")
		input_ = {"adjective": "funny", "topic": "cats"} # create a dictionary to store the corresponding input to placeholders in prompt template
		prompt.invoke(input_)
		
		Copied!
		
		Wrap Toggled!

	mplerinputs.	

Chat prompt templates	Used to format list of messages. These" t	```
		from langchain_core.prompts import ChatPromptTemplate
		
		prompt = ChatPromptTemplate.from_messages([
		  ("system", "You are a helpful assistant"),
		  ("user", "Tell me a joke about {topic}")
		])
		input_ = {"topic": "cats"}
		prompt.invoke(input_)
		``
		1 2 3 4 5 6 7

	e m p l a t e s " c o n s i s t o f a l i s t o f t e m p l a t e s t h e m s e l v e s .	Copied! Wrap Toggled!

Messagesplaceholder	This prompt template is responsible for adding a list of messages	

```
from langchain_core.prompts import MessagesPlaceholder

from langchain_core.messages import HumanMessage

prompt = ChatPromptTemplate.from_messages([

    ("system", "You are a helpful assistant"),

    MessagesPlaceholder("msgs")

])

input_ = {"msgs": [HumanMessage(content="What is the day after Tuesday?")]}

prompt.invoke(input_)
```

Copied!

Wrap Toggled!

	s s a g e s i n a p a r t i c u l a r p l a c e . B u t i f y o u w a n t t h e u s e r t o p a s	

	s i n a l i s t o f m e s s a g e s t h a t y o u w o u l d s l o t i n t o a p a r t i c u l a r	

	spot, the given codes nippetis helpful.	

Example selector	If you have many examples, you may need to select which ones	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21

to include in the prompt. The ExampleSelector is the class

```
from langchain_core.example_selectors import LengthBasedExampleSelector
from langchain_core.prompts import FewShotPromptTemplate, PromptTemplate
# Examples of a pretend task of creating antonyms.
examples = [
    {"input": "happy", "output": "sad"},
    {"input": "tall", "output": "short"},
    {"input": "energetic", "output": "lethargic"},
    {"input": "sunny", "output": "gloomy"},
    {"input": "windy", "output": "calm"},
]
example_prompt = PromptTemplate(
    input_variables=["input", "output"],
    template="Input: {input}\nOutput: {output}",
)
```

responsible for doing so.

```
example_selector = LengthBasedExampleSelector(
    examples=examples,
    example_prompt=example_prompt,
    max_length=25,  # The maximum length that the formatted examples should be.
)
dynamic_prompt = FewShotPromptTemplate(
    example_selector=example_selector,
    example_prompt=example_prompt,
    prefix="Give the antonym of every input",
    suffix="Input: {adjective}\nOutput:",
    input_variables=["adjective"],
)
```

Copied!

Wrap Toggled!

JSON parser	This output parser allows users to specify an arbitrary JS	

```
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
```

- from langchain_core.output_parsers import JsonOutputParser
- from langchain_core.pydantic_v1 import BaseModel, Field
- # Define your desired data structure.

ONschemaandqueryLLMsforoutputsthatconformtothatsc

```
class Joke(BaseModel):
    setup: str = Field(description="question to set up a joke")
    punchline: str = Field(description="answer to resolve the joke")
# And a query intented to prompt a language model to populate the data structure.
joke_query = "Tell me a joke."
# Set up a parser + inject instructions into the prompt template.
output_parser = JsonOutputParser(pydantic_object=Joke)
format_instructions = output_parser.get_format_instructions()
prompt = PromptTemplate(
    template="Answer the user query.\n{format_instructions}\n{query}\n",
    input_variables=["query"],
    partial_variables={"format_instructions": format_instructions},
)
chain = prompt | mixtral_llm | output_parser
chain.invoke({"query": joke_query})
```

Copied!

Wrap Toggled!

	hema.	
Commaseparatedlistparser	This output parser can be used when you want to retu	1 2 3 4 5 6 7 8 9 from langchain.output_parsers import CommaSeparatedListOutputParser output_parser = CommaSeparatedListOutputParser() format_instructions = output_parser.get_format_instructions() prompt = PromptTemplate(template="Answer the user query. {format_instructions}\nList five {subject}.", input_variables=["subject"], partial_variables={"format_instructions": format_instructions},

	rnalistofcommma-separateditems.	) chain = prompt | mixtral_llm | output_parser Copied! Wrap Toggled!
Document object	Containsinformati	1 2 3 4 5 6 7

on about some data in LangChain. It has two attributes: pag

8

```
from langchain_core.documents import Document

Document(page_content="""Python is an interpreted high-level general-purpose programming language.
                    Python's design philosophy emphasizes code readability with its notable use of significant indentation.""",
        metadata={
            'my_document_id' : 234234,
            'my_document_source' : "About Python",
            'my_document_create_time' : 1680013019
        })
```

Copied!

Wrap Toggled!

	e_content:str:This attribute holds the content of the do	

	c u m e n t . m e t a d a t a : d i c t : T h i s a t t r i b u t e c o n t a i n s a r b i t r	

	a r y m e t a d a t a a s s o c i a t e d w i t h t h e d o c u m e n t . I t c a n b e u s e d t	

	otrackvariousdetailssuchasthedocumentid,filename,	

	a n d s o o n .	
tex t_s plit ter	A t a h i g h l e v e l , t e x t s p l i t t e r s w o r k a s f o l l o w	1 2 3 • text_splitter = CharacterTextSplitter(chunk_size=200, chunk_overlap=20, separator="\n") # define chunk_size which is length of characters, and also separator. • chunks = text_splitter.split_documents(document) • print(len(chunks)) Copied! Wrap Toggled!

	s: • Split the text into small, semantically meaningful c	

	hunks(oftensentences). • Start combining these small c	

	hunksintoalargerchhunkuntilyourreachacertainsize(as	

	measuredbysomefunction). • Onceyoureachthatsize, ma	

	k e t h a t c h u n k i t s o w n p i e c e o f t e x t a n d s t a r t c r e a t i n g a n e w c	

	hunk with some overlap (to keep context between chunks).	

Embedding models	Embedding models are specifically designed to interface	```
		from ibm_watsonx_ai.metanames import EmbedTextParamsMetaNames
		embed_params = {
		EmbedTextParamsMetaNames.TRUNCATE_INPUT_TOKENS: 3,
		EmbedTextParamsMetaNames.RETURN_OPTIONS: {"input_text": True},
		}
		from langchain_ibm import WatsonxEmbeddings
		watsonx_embedding = WatsonxEmbeddings(
		model_id="ibm/slate-125m-english-rtrvr",
		``
		1 2 3 4 5 6 7 8 9 10 11 12

w
i
t
h
t
e
x
t
e
m
b
e
d
d
i
n
g
s
.
E
m
b
e
d
d
i
n
g
s
g
e
n
e
r
a
t
e
a
v
e
c
t
o
r
r
e
p
r
e

```
    url="https://us-south.ml.cloud.ibm.com",
    project_id="skills-network",
    params=embed_params,
)
```

Copied!

Wrap Toggled!

	s e n t a t i o n f o r a g i v e n p i e c e o f t e x t . T h i s i s a d v a n t a g e o u s a	

	s i t a l l o w s y o u t o c o n c e p t u a l i z e t e x t w i t h i n a v e c t o r s p a c e	

	.Consequently, you can perform operations such as semant	

	i c s e a r c h , w h e r e y o u i d e n t i f y p i e c e s o f t e x t t h a t a r e m o s t s	

	imilarwithinthevectorspace.	
Vector store-backed retriever	A retriever that uses a vav	1 2 3 • retriever = docsearch.as_retriever() • docs = retriever.invoke("Langchain")A vector store retriever is a retriever that uses a vector store to retrieve documents. It is a lightweight wrapper around the vector store class to make it conform to the retriever interface. It uses the search methods implemented by a vector store, like

	e c t o r s t o r e t o r e t r i e v e d o c u m e n t s . I t i s a l i g h t w e i g h t w r a	similarity search and MMR (Maximum marginal relevance), to query the texts in the vector store. • Since we've constructed a vector store docsearch, it's very easy to construct a retriever. Copied! Wrap Toggled!

	p p e r a r o u n d t h e v e c t o r s t o r e c l a s s t o m a k e i t c o n f o r m t o t h e	

	retrieve er interface. It uses the search methods implemen	

	tedbyavectorstore, likesimilaritysearchandMMR(maxi	

	mummarginalrelevance), toquerythetextsinthevectors	

	t o r e . S i n c e w e ' v e c o n s t r u c t e d a v e c t o r s t o r e d o c s e a r c h , i	

	t's very easy to construct a retriever.	
ChatMessageHistory class	One of the core ut	```1 2 3 4 5 from langchain.memory import ChatMessageHistory ```

	ilitycl assesunderpinningmost(ifnotall)memorymodul	• chat = mixtral_llm • history = ChatMessageHistory() • history.add_ai_message("hi!") • history.add_user_message("what is the capital of France?") Copied! Wrap Toggled!

	e s i s t h e C h a t M e s s a g e H i s t o r y c l a s s . T h i s s u p e r l i g h t w e i g	

	h t w r a p p e r p r o v i d e s c o n v e n i e n t m e t h o d s f o r s a v i n g H u m a n M	

	essages, AlMessages, and then fetching the mail.	

langchain.chains	This code snippet uses a LangChain library for building l	1
		2
		3
		4
		5
		6
		7
		8
		9
		from langchain.chains import LLMChain
		template = """Your job is to come up with a classic dish from the area that the users suggests.
		{location}
		YOUR RESPONSE:
		"""
		prompt_template = PromptTemplate(template=template, input_variables=['location'])
		# chain 1
		location_chain = LLMChain(llm=mixtral_llm, prompt=prompt_template, output_key='meal')

	a n g u a g e m o d e l a p p l i c a t i o n s , c r e a t i n g a c h a i n t o g e n e r a t e	• location_chain.invoke(input={'location':'China'}) Copied! Wrap Toggled!

	popular dishrecommendations based on the specified loca	

	tions. It also configures model inference settings for fu	

	rtherprocessing.	
Simple sequential chain	Sequential chain allow the output to	1 2 3 4 5 6 7 8 9 10 11 12 13

	f o n e L L M t o b e u s e d a s t h e i n p u t f o r a n o t h e r . T h i s a p p r o a c h i	14
		15
		16
		17
		18
		from langchain.chains import SequentialChain
		template = """Given a meal {meal}, give a short and simple recipe on how to make that dish at home.
		YOUR RESPONSE:
		"""
		prompt_template = PromptTemplate(template=template, input_variables=['meal'])
		# chain 2
		dish_chain = LLMChain(llm=mixtral_llm, prompt=prompt_template, output_key='recipe')
		template = """Given the recipe {recipe}, estimate how much time I need to cook it.
		YOUR RESPONSE:
		"""
		prompt_template = PromptTemplate(template=template, input_variables=['recipe'])

s
b
e
n
e
f
i
c
i
a
l
f
o
r
d
i
v
i
d
i
n
g
t
a
s
k
s
a
n
d
m
a
i
n
t
a
i
n
i
n
g
t
h
e
f
o
c
u
s

```
# chain 3
recipe_chain = LLMChain(llm=mixtral_llm, prompt=prompt_template,
output_key='time')
# overall chain
overall_chain = SequentialChain(chains=[location_chain, dish_chain,
recipe_chain],
                                input_variables=['location'],
                                output_variables=['meal', 'recipe', 'time'],
                                verbose= True)
```

Copied!

Wrap Toggled!

	of your LLM.	
load_summarize_chain	This code snippet uses LangChain library fo	1 2 3 4 • from langchain.chains.summarize import load_summarize_chain • chain = load_summarize_chain(llm=mixtral_llm, chain_type="stuff", verbose=False) • response = chain.invoke(web_data) • print(response['output_text'])n Copied! Wrap Toggled!

	rloadingandusingasummarizationchainwithhaspecificl	

	a n g u a g e m o d e l a n d c h a i n t y p e . T h i s c h a i n t y p e w i l l b e a p p l i	

	edtowebdatatoprintaresultingsummary.	

Text Classifier	Represents as implement text classifier that uses an embedding	```
		from torch import nn
		
		class TextClassifier(nn.Module):
		``

glayer, a hidden linear layer with a ReLU activation, and a

```
def __init__(self, num_classes,freeze=False):
    super(TextClassifier, self).__init__()
    self.embedding = nn.Embedding.from_pretrained(glove_embedding.vectors.to(device),freeze=freeze)
    # An example of adding additional layers: A linear layer and a ReLU activation
    self.fc1 = nn.Linear(in_features=100, out_features=128)
    self.relu = nn.ReLU()
    # The output layer that gives the final probabilities for the classes
    self.fc2 = nn.Linear(in_features=128, out_features=num_classes)
def forward(self, x):
    # Pass the input through the embedding layer
    x = self.embedding(x)
    # Here you can use a simple mean pooling
    x = torch.mean(x, dim=1)
    # Pass the pooled embeddings through the additional layers
    x = self.fc1(x)
    x = self.relu(x)
    return self.fc2(x)
```

	n o u t p u t l i n e a r l a y e r . T h e c o n s t r u c t o r t a k e s t h e f o l l o w i n	Copied! Wrap Toggled!

	gargument: num_class: The number of classes to classify	

	.freeze: Whether to freeze the embedding layer.	

Train the model	This code sniippet outlines the function to train a machine	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21

learningmodelusingPyTorch.Thisfunctiontrainsthemo

```
def train_model(model, optimizer, criterion, train_dataloader, valid_dataloader, epochs=100, model_name="my_modeldrop"):
    cum_loss_list = []
    acc_epoch = []
    best_acc = 0
    file_name = model_name
    for epoch in tqdm(range(1, epochs + 1)):
        model.train()
        cum_loss = 0
        for _, (label, text) in enumerate(train_dataloader):
            optimizer.zero_grad()
            predicted_label = model(text)
            loss = criterion(predicted_label, label)
            loss.backward()
            torch.nn.utils.clip_grad_norm_(model.parameters(), 0.1)
            optimizer.step()
```

delovera specified number of epochs, tracks them, and eva

```
            cum_loss += loss.item()

        #print("Loss:", cum_loss)

        cum_loss_list.append(cum_loss)

        acc_val = evaluate(valid_dataloader, model, device)

        acc_epoch.append(acc_val)

        if acc_val > best_acc:

            best_acc = acc_val

            print(f"New best accuracy: {acc_val:.4f}")

            #torch.save(model.state_dict(), f"{model_name}.pth")

    #save_list_to_file(cum_loss_list, f"{model_name}_loss.pkl")

    #save_list_to_file(acc_epoch, f"{model_name}_acc.pkl")
```

Copied!

Wrap Toggled!

	luatesthheperformanceonthedataset.	

llm_model	This code specifies the function 'llm_model' for generat	[redacted]

ingtextusingthelanguagemodelfromthemistral.aiplat

```
def llm_model(prompt_txt, params=None):
    model_id = 'mistralai/mixtral-8x7b-instruct-v01'
    default_params = {
        "max_new_tokens": 256,
        "min_new_tokens": 0,
        "temperature": 0.5,
        "top_p": 0.2,
        "top_k": 1
    }
    if params:
        default_params.update(params)
```

form, specifically the 'mistral-8x7b-instruct-v01' mod

```
parameters = {
    GenParams.MAX_NEW_TOKENS: default_params["max_new_tokens"],  # this controls the maximum number of tokens in the generated output
    GenParams.MIN_NEW_TOKENS: default_params["min_new_tokens"], # this controls the minimum number of tokens in the generated output
    GenParams.TEMPERATURE: default_params["temperature"], # this randomness or creativity of the model's responses
    GenParams.TOP_P: default_params["top_p"],
    GenParams.TOP_K: default_params["top_k"]
}
credentials = {
    "url": "https://us-south.ml.cloud.ibm.com"
}
project_id = "skills-network"
model = Model(
    model_id=model_id,
    params=parameters,
    credentials=credentials,
    project_id=project_id
```

el. The function helps in customizing generating parameter

```
    )

    mixtral_llm = WatsonxLLM(model=model)

    response  = mixtral_llm.invoke(prompt_txt)

    return response
```

Copied!

Wrap Toggled!

	ersandinteracts with IBM Watson's machine learning serv	

	ices.	
Zero-shot prompt	Zero-shot learning is crucial for testing a model'	1 2 3 4 5 6 7 prompt = """Classify the following statement as true or false: 'The Eiffel Tower is located in Berlin.' Answer: """ response = llm_model(prompt, params) print(f"prompt: {prompt}\n") print(f"response : {response}\n") Copied! Wrap Toggled!

	s a b i l i t y t o a p p l y i t s p r e - t r a i n e d k n o w l e d g e t o n e w , u n s e e	

	n t a s k s w i t h o u t a d d i t i o n a l t r a i n i n g . T h i s c a p a b i l i t y i s v	

	a l u a b l e f o r g a u g i n g t h e m o d e l' s g e n e r a l i z a t i o n s k i l l s .	

One-shot prompt	One-shot learning example where the model is given a single	

```
1
2
3
4
5
6
7
8
9
10
11
12
13
params = {
    "max_new_tokens": 20,
    "temperature": 0.1,
}
prompt = """Here is an example of translating a sentence from English to French:
        English: “How is the weather today?”
        French: “Comment est le temps aujourd'hui?”
```

e
e
x
a
m
p
l
e
t
o
h
e
l
p
g
u
i
d
e
i
t
s
t
r
a
n
s
l
a
t
i
o
n
f
r
o
m
E
n
g
l
i
s
h
t
o
F
r
e

```
        Now, translate the following sentence from English to French:

        English: “Where is the nearest supermarket?”

    """

    response = llm_model(prompt, params)

    print(f"prompt: {prompt}\n")

    print(f"response : {response}\n")
```

Copied!

Wrap Toggled!

	nch.Theprompt provides a sample translation pairing, "H	

	owisthheweathertoday?"translatedto"Commentestletem	

	psaujourd'hui?"This example serves as a guide for the mod	

	e l t o u n d e r s t a n d t h e t a s k c o n t e x t a n d d e s i r e d f o r m a t . T h e m	

	odelisthentaskedwithtranslatinganewsentence,"Wher	

	e i s t h e n e a r e s t s u p e r m a r k e t ? " w i t h o u t f u r t h e r g u i d a n c	

	e.	
Few-shot prompt	This code sniippet classifies sentiments using a few-sh	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 • #parameters `max_new_tokens` to 10, which constrains the model to generate brief responses

o
t
l
e
a
r
n
i
n
g
a
p
p
r
o
a
c
h
.
T
h
e
p
r
o
m
p
t
i
n
c
l
u
d
e
s
v
a
r
i
o
u
s
e
x
a
m
p
l

```
params = {
  "max_new_tokens": 10,
}
prompt = """Here are few examples of classifying emotions in statements:
        Statement: 'I just won my first marathon!'
        Emotion: Joy
        Statement: 'I can't believe I lost my keys again.'
        Emotion: Frustration
        Statement: 'My best friend is moving to another country.'
        Emotion: Sadness
        Now, classify the emotion in the following statement:
        Statement: 'That movie was so scary I had to cover my eyes.'
"""
response = llm_model(prompt, params)
print(f"prompt: {prompt}\n")
print(f"response : {response}\n")
```

Copied!

Wrap Toggled!

	e s w h e r e s t a t e m e n t s a r e a s s o c i a t e d w i t h t h e i r r e s p e c t i v e	

	emotions.	

Chain-of-thought (CoT) prompting	The Chain-of-Thought (CoT) prompting techn	```
		
		params = {
		"max_new_tokens": 512,
		"temperature": 0.5,
		}
		``

i
q
u
e
,
d
e
s
i
g
n
e
d
t
o
g
u
i
d
e
t
h
e
m
o
d
e
l
t
h
r
o
u
g
h
a
s
e
q
u
e
n
c
e
o
f
r
e
a

```
prompt = """Consider the problem: 'A store had 22 apples. They sold 15 apples today and got a new delivery of 8 apples.
        How many apples are there now?'
        Break down each step of your calculation
"""
response = llm_model(prompt, params)
print(f"prompt: {prompt}\n")
print(f"response : {response}\n")
```

Copied!

Wrap Toggled!

	s o n i n g s t e p s t o s o l v e a p r o b l e m . T h e C o T t e c h n i q u e i n v o l v	

	e s s t r u c t u r i n g t h e p r o m p t b y i n s t r u c t i n g t h e m o d e l t o " B r e	

	a k d o w n e a c h s t e p o f y o u r c a l c u l a t i o n . " T h i s e n c o u r a g e s t h	

	e m o d e l t o i n c l u d e e x p l i c i t r e a s o n i n g s t e p s , m i m i c k i n g h u	

	man-likeproblem-solvingprocesses.	
Self-consistency	This codesnipp	1 2 3 4 5 6

	e t d e t e r m i n e s t h e c o n s i s t e n t r e s u l t f o r a g e - r e l a t e d p r o b	7
		8
		9
		params = {
		"max_new_tokens": 512,
		}
		prompt = """When I was 6, my sister was half of my age. Now I am 70, what age is my sister?
		Provide three independent calculations and explanations, then determine the most consistent result.
		"""
		response = llm_model(prompt, params)
		print(f"prompt: {prompt}\n")
		print(f"response : {response}\n")
		
		Copied!
		
		Wrap Toggled!

	lemsandgenerates multiple responses. The 'params' dict	

	ionaryspecifiesthemaximumnumberoftokenstogenerate	

	responses.	
Prompt template	A key concept in Lang Chain, it helps to trans	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16

Lateuserinputandparametersintoinstructionsforalan

```
model_id = 'mistralai/mixtral-8x7b-instruct-v01'

parameters = {
    GenParams.MAX_NEW_TOKENS: 256,  # this controls the maximum number of tokens in the generated output
    GenParams.TEMPERATURE: 0.5, # this randomness or creativity of the model's responses
}
credentials = {
    "url": "https://us-south.ml.cloud.ibm.com"
}
project_id = "skills-network"
model = Model(
    model_id=model_id,
    params=parameters,
    credentials=credentials,
    project_id=project_id
)
mixtral_llm = WatsonxLLM(model=model)
mixtral_llm
```

	guagemodel.Thiscanbeusedtoguideamodel'sresponse,he	Copied! Wrap Toggled!

	l p i n g i t u n d e r s t a n d t h e c o n t e x t a n d g e n e r a t e r e l e v a n t a n d	

	coherent language-based output.	
Text summarization	Text summarizationa	1 2 3 4 5 6 7 8

generated to help summarize the content you provide to

```
content = """
    The rapid advancement of technology in the 21st century has transformed various industries, including healthcare, education, and transportation.
    Innovations such as artificial intelligence, machine learning, and the Internet of Things have revolutionized how we approach everyday tasks and complex problems.
    For instance, AI-powered diagnostic tools are improving the accuracy and speed of medical diagnoses, while smart transportation systems are making cities more efficient and reducing traffic congestion.
    Moreover, online learning platforms are making education more accessible to people around the world, breaking down geographical and financial barriers.
    These technological developments are not only enhancing productivity but also contributing to a more interconnected and informed society.
```

the LLM. You can store the content to be summarized in a variable

```
"""
template = """Summarize the {content} in one sentence.
"""
prompt = PromptTemplate.from_template(template)
llm_chain = LLMChain(prompt=prompt, llm=mixtral_llm)
response = llm_chain.invoke(input = {"content": content})
print(response["text"])
```

Copied!

Wrap Toggled!

	a b l e , a l l o w i n g f o r r e p e a t e d u s e o f t h e p r o m p t .	

Question answering	An agent that enables the LLM to learn from the provided con	1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 • content = """ • The solar system consists of the Sun, eight planets, their moons, dwarf planets, and smaller objects like asteroids and comets. • The inner planets—Mercury, Venus, Earth, and Mars—are rocky and solid.

```
        The outer planets—Jupiter, Saturn, Uranus, and Neptune—are much larger and gaseous.
    """
question = "Which planets in the solar system are rocky and solid?"
template = """
        Answer the {question} based on the {content}.
        Respond "Unsure about answer" if not sure about the answer.
        Answer:
"""
prompt = PromptTemplate.from_template(template)
output_key = "answer"
llm_chain = LLMChain(prompt=prompt, llm=mixtral_llm, output_key=output_key)
response = llm_chain.invoke(input = {"question":question ,"content": content})
print(response["answer"])
```

Copied!

Wrap Toggled!

tentandanswerquestionsbasedonwhatithaslearned.Occ

	asionally, if the LLM does not have sufficient information	

	n, it might generate speculative answer. To manage this,	

	you'lls pecifically instructitto responds with "Unsureab	

	o u t t h e a n s w e r " i f i t i s u n c e r t a i n a b o u t t h e c o r r e c t r e s p o n	

	se.	
Code generation	Agent that is designed to generate SQL queries based	1 2 3 4 5 6 7 8 9 10 11 12 13 description = """ Retrieve the names and email addresses of all customers from the 'customers' table who have made a purchase in the last 30 days. The table 'purchases' contains a column 'purchase_date' """ template = """

e
d
o
n
g
i
v
e
n
d
e
s
c
r
i
p
t
i
o
n
s
.
I
t
i
n
t
e
r
p
r
e
t
s
t
h
e
r
e
q
u
i
r
e
m
e
n
t
s

```
        Generate an SQL query based on the {description}

        SQL Query:

"""

prompt = PromptTemplate.from_template(template)

output_key = "query"

llm_chain = LLMChain(prompt=prompt, llm=mixtral_llm,

output_key=output_key)

response = llm_chain.invoke(input = {"description":description})

print(response["query"])
```

Copied!

Wrap Toggled!

	fromyourinputandtranslatesthemintoexecutableSQLco	

	de.	
Role playing	Configures the LLM to assume specific roles as defin	``` role = """ game master """ tone = "engaging and immersive" template = """ You are an expert {role}. I have this question {question}. I would like our conversation to be {tone}. Answer: ```

edbyus,enablingittofollowpredeterminedrulesandbeh

```
"""
prompt = PromptTemplate.from_template(template)
output_key = "answer"
llm_chain = LLMChain(prompt=prompt, llm=mixtral_llm,
output_key=output_key)
```

Copied!

Wrap Toggled!

	a v e l i k e a t a s k - o r i e n t e d c h a t b o t .	
cla ss _n am es	T h i s c o d e s n i p p e t m a p	1 2 • class_names = {0: "negative", 1: "positive"} • class_names Copied! Wrap Toggled!

	s n u m e r i c a l l a b e l s t o t h e i r c o r r e s p o n d i n g t e x t u a l d e s c r i	

	p t i o n s t o c l a s s i f y t a s k s . T h i s c o d e h e l p s i n m a c h i n e l e a r n	

	ingtointerpretheoutputmodel,wherethemodel'spredic	

	tionsarenumericalandshouldbepresentedinamorehuman	

	-readableformat.	
read_and_split_text	Involves opening the file, reading i	1 2 3 4 5 6 7 8 9 10 11 • def read_and_split_text(filename): • with open(filename, 'r', encoding='utf-8') as file:

t s c o n t e n t s , a n d s p l i t t i n g t h e t e x t i n t o i n d i v i d u a l p a r a g

```
        text = file.read()

    # Split the text into paragraphs (simple split by newline characters)

    paragraphs = text.split('\n')

    # Filter out any empty paragraphs or undesired entries

    paragraphs = [para.strip() for para in paragraphs if len(para.strip()) > 0]

    return paragraphs

# Read the text file and split it into paragraphs

paragraphs = read_and_split_text('companyPolicies.txt')

paragraphs[0:10]
```

Copied!

Wrap Toggled!

	r a p h s . E a c h p a r a g r a p h r e p r e s e n t s a s e c t i o n o f t h e c o m p a n y	

	policies. You can also filter out any empty paragraphs to c	

	leany ourdat aset.	
encode_contexts	This codesnippet encodes a list of te	1 2 3 4 5 6 7 8 9 10 • def encode_contexts(text_list): • # Encode a list of texts into embeddings • embeddings = []

xtsintoembeddingsusingcontent_tokenizerandcontext

```
    for text in text_list:
        inputs = context_tokenizer(text, return_tensors='pt', padding=True, truncation=True, max_length=256)
        outputs = context_encoder(**inputs)
        embeddings.append(outputs.pooler_output)
    return torch.cat(embeddings).detach().numpy()
# you would now encode these paragraphs to create embeddings.
context_embeddings = encode_contexts(paragraphs)
```

Copied!

Wrap Toggled!

	_encoder. This code helps iterate through each text in the	

	input list, tokenizes and encodes it, and then appends the	

	pooler_output to the embedding list. The resulting embe	

	ddingsgetstoreedinthecontext_embeddingsvariablesan	

	dgenerateembeddingsfromtextdataforvariousnaturalll	

	a n g u a g e p r o c e s s i n g (N L P) a p p l i c a t i o n s .	

import faiss	FAISS (FacebookAISimilaritySearch) is an efficient lib	1
		2
		3
		4
		5
		6
		7
		import faiss
		
		# Convert list of numpy arrays into a single numpy array
		embedding_dim = 768 # This should match the dimension of your embeddings
		context_embeddings_np = np.array(context_embeddings).astype('float32')
		# Create a FAISS index for the embeddings
		index = faiss.IndexFlatL2(embedding_dim)
		index.add(context_embeddings_np) # Add the context embeddings to the index
		
		Copied!
		
		Wrap Toggled!

	rary developed by Facebook for similarity search and clus	

	t e r i n g o f d e n s e v e c t o r s . F A I S S i s d e s i g n e d f o r f a s t s i m i l a	

	r i t y s e a r c h , w h i c h i s p a r t i c u l a r l y v a l u a b l e w h e n d e a l i n g	

	with large data sets. It is highly suitable for tasks in nat	

	u r a l l a n g u a g e p r o c e s s i n g w h e r e r e t r i e v a l s p e e d i s c r i t i c	

	al. Ite ffectivelyhandleslargevolumesoffdata, maintai	

	ningperformanceevenasdatasetsizesincrease.	

search_relevant_contexts	This code snippet is useful in search ing relevant context	```
		def search_relevant_contexts(question, question_tokenizer, question_encoder, index, k=5):
		"""
		Searches for the most relevant contexts to a given question.
		Returns:
		tuple: Distances and indices of the top k relevant contexts.
		"""
		# Tokenize the question
		``

s
f
o
r
a
g
i
v
e
n
q
u
e
s
t
i
o
n
.
I
t
t
o
k
e
n
i
z
e
s
t
h
e
q
u
e
s
t
i
o
n
u
s
i
n
g
t
h
e

```
    question_inputs = question_tokenizer(question, return_tensors='pt')

    # Encode the question to get the embedding

    question_embedding =

question_encoder(**question_inputs).pooler_output.detach().numpy()

    # Search the index to retrieve top k relevant contexts

    D, I = index.search(question_embedding, k)

    return D, I
```

Copied!

Wrap Toggled!

	question_tokenizer,encodesthequestionusingquestio	

	n_encoder, and search an index for retrieving the relev	

	antcontextbasedonquestionembedding.	
generate_answer_without_c	This code snip	1 2 3 4 5

ont
ext

p
e
t
g
e
n
e
r
a
t
e
s
r
e
s
p
o
n
s
e
s
u
s
i
n
g
t
h
e
e
n
t
e
r
e
d
p
r
o
m
p
t
w
i
t
h
o
u
t

```
6
7
8
def generate_answer_without_context(question):
    # Tokenize the input question
    inputs = tokenizer(question, return_tensors='pt', max_length=1024, truncation=True)
    # Generate output directly from the question without additional context
    summary_ids = model.generate(inputs['input_ids'], max_length=150, min_length=40, length_penalty=2.0, num_beams=4, early_stopping=True)
    # Decode and return the generated text
    answer = tokenizer.decode(summary_ids[0], skip_special_tokens=True)
    return answer
```

Copied!

Wrap Toggled!

	requiringadditionalcontext.Ittokenizestheinputque	

	stionsusingthetokenizer,generatestheoutputtextusi	

	ngthemodel,anddecodesthegeneratedtextto obtainthea	

	n s w e r .	

Generating answers with DPR contexts	Answers are generated when the model utilizes co	1
		2
		3
		4
		5
		6
		7
		def generate_answer(contexts):
		  # Concatenate the retrieved contexts to form the input to BART
		  input_text = ' '.join(contexts)
		  inputs = tokenizer(input_text, return_tensors='pt', max_length=1024, truncation=True)
		  # Generate output using BART
		  summary_ids = model.generate(inputs['input_ids'], max_length=150, min_length=40, length_penalty=2.0, num_beams=4, early_stopping=True)
		  return tokenizer.decode(summary_ids[0], skip_special_tokens=True)

	n t e x t s r e t r i e v e d v i a D P R , w h i c h a r e e x p e c t e d t o e n h a n c e t h	Copied! Wrap Toggled!

	e a n s w e r ' s r e l e v a n c e a n d d e p t h :	

aggregate_embeddings function	The function aggregate_embeddings takes a token indices a	[redacted]

n d t h e i r c o r r e s p o n d i n g a t t e n t i o n m a s k s , a n d u s e s a B E R T m o

```
def aggregate_embeddings(input_ids, attention_masks, bert_model=bert_model):
    """
    Converts token indices and masks to word embeddings, filters out zero-padded embeddings,
    and aggregates them by computing the mean embedding for each input sequence.
    """
    mean_embeddings = []
    # Process each sequence in the batch
    print('number of inputs',len(input_ids))
    for input_id, mask in tqdm(zip(input_ids, attention_masks)):
        input_ids_tensor = torch.tensor([input_id]).to(DEVICE)
        mask_tensor = torch.tensor([mask]).to(DEVICE)
        with torch.no_grad():
            # Obtain the word embeddings from the BERT model
            word_embeddings = bert_model(input_ids_tensor, attention_mask=mask_tensor)[0].squeeze(0)
            # Filter out the embeddings at positions where the mask is zero
            valid_embeddings_mask=mask_tensor[0] != 0
```

delto convert the set to keens into word embeddings. It then f

```
        valid_embeddings = word_embeddings[valid_embeddings_mask,:]
        # Compute the mean of the filtered embeddings
        mean_embedding = valid_embeddings.mean(dim=0)
        mean_embeddings.append(mean_embedding.unsqueeze(0))
    # Concatenate the mean embeddings from all sequences in the batch
    aggregated_mean_embeddings = torch.cat(mean_embeddings)
    return aggregated_mean_embeddings
```

Copied!

Wrap Toggled!

	i l t e r s o u t t h e e m b e d d i n g s f o r z e r o - p a d d e d t o k e n s a n d c o m p	

	u t e s t h e m e a n e m b e d d i n g f o r e a c h s e q u e n c e . T h i s h e l p s i n r e	

	ducing the dimensionality of the data while retaining the e	

	most important information from the embeddings.	
text_to_	Desig	1 2

emb	needtoconvertalistoftextstringsintotheircorrespond	3 • def text_to_emb(list_of_text,max_input=512): • data_token_index = tokenizer.batch_encode_plus(list_of_text, add_special_tokens=True,padding=True,truncation=True,max_length=max_input) question_embeddings=aggregate_embeddings(data_token_index['input_ids'], data_token_index['attention_mask']) • return question_embeddings Copied! Wrap Toggled!

	ingembeddingsusingapre-definedtokenizer.	

process_song	Convert both the predefined appropriate newsquestions a	

```
import re

def process_song(song):
    # Remove line breaks from the song
    song_new = re.sub(r'[\n]', ' ', song)
    # Remove single quotes from the song
    song_new = [song_new.replace("\'", "")]
    return song_new
```

Copied!

Wrap Toggled!

	n d t h e s o n g l y r i c s i n t o " R A G e m b e d d i n g s " a n d m e a s u r e t h e s i	

	m i l a r i t y b e t w e e n t h e m t o d e t e r m i n e t h e a p p r o p r i a t e n e s	

	s.	
RAG_QA	This code implements question-answering using	1 2 3 4 5 6 7 8 9 10 11 12 • def RAG_QA(embeddings_questions, embeddings, n_responses=3): • # Calculate the dot product between the question embeddings and the provided embeddings (transpose of the second matrix for proper alignment). • dot_product = embeddings_questions @ embeddings.T • # Reshape the dot product results to a 1D tensor for easier processing.

gquestionembeddingsandprovidesembeddings.Ithelpsr

```
dot_product = dot_product.reshape(-1)
# Sort the indices of the dot product results in descending order (setting descending to False should be True for typical similarity tasks).
sorted_indices = torch.argsort(dot_product, descending=True)
# Convert sorted indices to a list for easier iteration.
sorted_indices = sorted_indices.tolist()
# Print the top 'n_responses' responses from the sorted list, which correspond to the highest dot product values.
for index in sorted_indices[:n_responses]:
    print(yes_responses[index])
```

Copied!

Wrap Toggled!

	e s h a p e t h e r e s u l t s f o r p r o c e s s i n g , s o r t i n g t h e i n d i c e s i n	

	descending order, and printing the top 'n-responses' bas	

	edonthehighestdotproductvalues.	
model_name_or_path	Thiscodesnippetd	1 2 3 4 5 6 7

defines the model name to 'gpt2' and initializes the tokena

```
8
9
10
# Define the model name or path
model_name_or_path = "gpt2"
# Initialize tokenizer and model
tokenizer = GPT2Tokenizer.from_pretrained(model_name_or_path,
use_fast=True)
model =
GPT2ForSequenceClassification.from_pretrained(model_name_or_path
, num_labels=1)
# Add special tokens if necessary
tokenizer.pad_token = tokenizer.eos_token
model.config.pad_token_id = model.config.eos_token_id
# Define the maximum length
max_length = 1024
```

Copied!

Wrap Toggled!

	ndmodelusingtheGPT-2model.Inthiscode,addspecialto	

	kensforpaddingbykeepingthemaximumsequencelengthto	

	1024.	
add_combined_columns	This code snippet combines the prompt with chosen	1 2 3 4 5 6 7 8 9 10 • # Define a function to combine 'prompt' with 'chosen' and 'rejected' responses • def add_combined_columns(example): • # Combine 'prompt' with 'chosen' response, formatting it with "Human:" and "Assistant:" labels • example['prompt_chosen'] = "\n\nHuman: " + example["prompt"] + "\n\nAssistant: " + example["chosen"]

andrejectedresponsesinadatasetexample. Itcombinesw

```
    # Combine 'prompt' with 'rejected' response, formatting it with "Human:" and "Assistant:" labels
    example['prompt_rejected'] = "\n\nHuman: " + example["prompt"] + "\n\nAssistant: " + example["rejected"]
    # Return the modified example
    return example
# Apply the function to each example in the 'train' split of the dataset
dataset['train'] = dataset['train'].map(add_combined_columns)
```

Copied!

Wrap Toggled!

	i t h t h e ' H u m a n : ' a n d ' A s s i s t a n t : ' f o r c l a r i t y . T h i s f u n c t	

	i o n m o d i f i e s e a c h e x a m p l e i n t h e ' t r a i n ' s p l i t t h e d a t a s e t	

	by creating new columns' prompt_chosen' and' prompt_rej	

	ected'withthecombinedtext.	
Retrieval QA	This codesnippetcreate	1 2 3 4 • qa = RetrievalQA.from_chain_type(llm=flan_ul2_llm, • chain_type="stuff", retriever=docsearch.as_retriever(), return_source_documents=False) • query = "what is mobile policy?"

	s a n e x a m p l e f o r ' R e t r i e v a l Q A ' u s i n g a l a n g u a g e m o d e l a n d d	• qa.invoke(query)

	o c u m e n t r e t r i e v e r .	